P

Photo Finder Studio

Project documentation

Requirements · Architecture · How it works · Database design

What it is

A web app for photography companies. Guests find their own photos from an event with a selfie; the studio's client gets a private gallery of every photo, can find people, search by description and choose album photos; and the studio runs its bookings, billing, tasks, equipment and team in the same place.

What this document covers

The requirements, the architecture and deployment, how each main flow works (with diagrams), the database design with entity-relationship diagrams and a full data dictionary, security, the API, testing and how to run it.

VERSION	Code as of 22 October 2026: commit a8bc0fc plus the guest PIN and search-by-description work
PREPARED BY	Aniruth
CODEBASE	Python 3 · FastAPI · SQLite · OpenCV · onnxruntime · vanilla JavaScript
SCOPE	The multi-company studio app (python -m studio). The single-folder app.py is described in README.md.

Contents

1. Overview . . . 3

Who uses it . . . 3

Project at a glance . . . 3

2. Requirements . . . 3

2.1 Functional requirements . . . 3

2.2 Non-functional requirements . . . 5

2.3 System requirements . . . 5

3. Architecture . . . 5

3.1 Modules . . . 6

3.2 Technology . . . 7

3.3 Deployment . . . 7

4. How it works . . . 8

4.1 From upload to searchable photo . . . 8

4.2 A guest finds their photos . . . 9

4.3 The client gallery . . . 10

4.4 Search by description . . . 10

4.5 Signing in, roles and company scoping . . . 11

4.6 The tasks board . . . 12

4.7 The studio's day-to-day workflow . . . 12

5. Database design . . . 12

5.1 Entity-relationship diagrams . . . 13

5.2 Design rules . . . 15

5.3 Data dictionary . . . 15

6. Security and privacy . . . 24

7. API overview . . . 25

8. Testing . . . 25

9. Running it . . . 26

10. Known limitations and next steps . . . 26

1. Overview

Photo Finder Studio is a web application for photography companies. A studio signs in, adds its clients, creates an event for each shoot and uploads the photos. Every event gets one link and QR code: a guest opens it on a phone, takes a selfie and sees only the photos they appear in. The studio's client (the couple, the company) gets a second, private link with every photo, where they can find people, search by description and choose photos for the album.

Around that core the studio runs its business in the same app: a calendar of bookings with payments, estimates and invoices as PDFs, a project board of tasks (including work given to outside vendors), an equipment inventory, a team with roles, and an overview page that pulls it together. Each company only ever sees its own data.

Who uses it

User	Uses	Needs an account?
Studio owner and team	The dashboard: clients, events, uploads, sharing, calendar, billing, tasks, inventory, team, settings.	Yes (email + password or Google). Role decides what they may change.
Guest	The guest page /e/<event> on a phone: selfie, their photos, downloads.	No. The link (and an optional PIN).
Client	The client gallery /c/<link>: every photo, find a person, search by description, album selection.	No. The private link (and an optional PIN).
Vendor	An upload link /v/<link> for one event (second shooter, drone, video team).	No. The upload link.
Camera	FTP or FTPS upload straight into one event, if the platform admin allowed it.	A per-event camera login.
Platform admin	The product owner's /admin: every company, usage, plans, suspension, activity log.	Yes, granted from the command line.

Project at a glance

Item	Value
Code	Python back end (25 files in studio/ plus faces.py) and 6 HTML pages with plain JavaScript; about 9,900 lines
HTTP routes	139 (pages, JSON API, files)
Database	SQLite, 24 tables, schema version 23 (applied automatically at start)
AI on the server	Face matching (YuNet + SFace, OpenCV) and search by description (CLIP ViT-B/32, onnxruntime). No cloud AI, no per-use cost.
Tests	174 automated tests (pytest), all passing; 2 more need a folder of sample faces
Runs on	Windows, macOS or Linux with Python 3.9 to 3.13; a Dockerfile and docker-compose.yml for servers

2. Requirements

2.1 Functional requirements

What the system does, grouped by area. Each one is implemented in the code described in section 3.

Accounts and team

ID	Requirement
FR-01	A photography company signs up with email and password, or Google. Each sign-up creates a separate company.

ID	Requirement
FR-02	The owner or an admin invites team members by email with a role: Owner, Admin, Manager, Photographer or Viewer. Invitations are one-time and expire after 7 days.
FR-03	A user who forgot their password gets a one-time reset link by email, valid 60 minutes.
FR-04	The studio sets its name, brand colour, logo (with an in-browser editor), the look of guest and client pages (dark, light, automatic) and its time zone.

Clients, events and uploads

ID	Requirement
FR-05	The studio keeps clients (name, email, phone, notes) and creates events under them (name, date, subtitle).
FR-06	An event can be published or hidden, given an expiry date, have guest downloads switched on or off, and have its face-matching strictness tuned.
FR-07	Photos are uploaded by drag and drop, by choosing files or a whole folder (JPG, PNG, WebP, HEIC, up to 60 MB each). Exact duplicates are skipped and plan limits are enforced.
FR-08	Cameras can upload over FTP or FTPS into an event, once the platform admin allows it for that event.
FR-09	Vendors get an upload link per event, need no account, and their files are tagged with their name.
FR-10	Videos can be uploaded to an event; they appear in the client gallery only.

Sharing

ID	Requirement
FR-11	Each event has a guest link, QR code and WhatsApp share. A guest takes a selfie and sees only the photos they appear in, with single downloads and Download all (zip).
FR-12	The guest link can carry a 4 to 8 digit PIN; changing it locks out every phone that used the old one.
FR-13	The client gallery is a private link to every photo, with an optional PIN, a downloads switch, a New link button and a view count.
FR-14	In the client gallery, Find a person filters to one person's photos from a selfie or any photo of them.
FR-15	Search by description finds photos from typed words ("group photo", "mehendi") and offers chips for this event's common moments; also on the studio's Photos card.
FR-16	The client selects photos for the album with an optional note; the studio sees the list, copies file names, downloads the originals and locks it.

Running the business

ID	Requirement
FR-17	A calendar of bookings: dates and times, the client, own client or Expost, the agreed amount and what has been received.
FR-18	Estimates and invoices, numbered per company (EST-0001, INV-0001), as PDFs; an estimate converts to an invoice.
FR-19	A project board of tasks: stages, status, deadline, people, tags, checklist, comments and history; a task can be given to a vendor with their number, the work, the agreed amount and what was paid.
FR-20	An equipment inventory: items, check-out to a team member for an event with a due date, check-in, maintenance, history and CSV export.
FR-21	An Overview page: what needs attention, my tasks, what is coming up, money to collect and owed, equipment out.

Platform

ID	Requirement
FR-22	The platform admin sees every company with its usage, changes plans, suspends and re-enables, signs a company out everywhere, allows camera upload, and has an activity log.
FR-23	Plans limit events, photos, storage and team seats (Free, Pro, Studio).

ID	Requirement
FR-24	Privacy controls: delete an event's face data, event expiry, turn off search by description, delete the whole company account.

2.2 Non-functional requirements

Area	Requirement and how it is met
Security	Passwords hashed with scrypt; session and reset tokens stored only as hashes; httpOnly cookies; a request header blocks cross-site form posts; rate limits on sign-in, searches, PIN tries and zips; every dashboard query scoped to the signed-in company.
Privacy	Selfies are processed in memory and never stored; only a count of searches is kept. All AI runs on the studio's server; no photo is sent to an outside AI service. Face data can be deleted per event; search-by-description data is deleted when the setting is turned off.
Performance	Uploads return as soon as the original is stored; processing runs in the background. About 0.5 photos per second per core for 24-megapixel photos (faster for phone photos). Guest search about 0.1 s even with 200,000 faces. Search by description: 101 photos indexed in about a minute on an 8-core PC, search is instant.
Reliability	Work in progress resumes after a restart; a crashed worker fails only its photo and the pool restarts; schema migrations run automatically and in order.
Scalability	One process with SQLite (WAL) suits a few dozen active studios. Storage is behind a small interface so S3 or R2 can replace the local disk.
Usability	Guest and client pages are built for phones first, follow the studio's brand colour and logo, and offer dark and light themes.
Maintainability	Small modules, one per feature, each mounted onto the app; 174 automated tests; plain JavaScript with no build step.

2.3 System requirements

Item	Requirement
Python	3.9 to 3.13 (the Docker image uses 3.12)
Python packages	fastapi, uvicorn, python-multipart, numpy, opencv-python-headless, pillow, pillow-heif, dlib-bin, qrcode, cryptography, pyopenssl, pyftpdlib, reportlab, tzdata, onnxruntime, tokenizers (requirements.txt)
AI models	Face models are in models/ (about 70 MB). The CLIP model for search by description (156 MB) is downloaded once with python -m studio get-scene-model.
Hardware	Face processing is the heavy part: about 35 minutes for a 3,000-photo wedding of 24 MP files on 4 cores, scaling with cores. Disk for the originals plus their thumbnail and web copies.
Network	Port 8000 (or behind an HTTPS reverse proxy on 443); optionally FTP 2121 and passive ports 60000-60049 for cameras.
Optional services	SMTP server for emails (otherwise printed to the console), Google OAuth client for Google sign-in.

3. Architecture

The whole system is one Python process: a FastAPI web application with background workers, a SQLite database and a folder of files. There is no separate front-end build: the server sends HTML pages with plain JavaScript, which then talk to the JSON API. Heavy work (photo processing) runs in a pool of worker processes so the web server stays responsive.

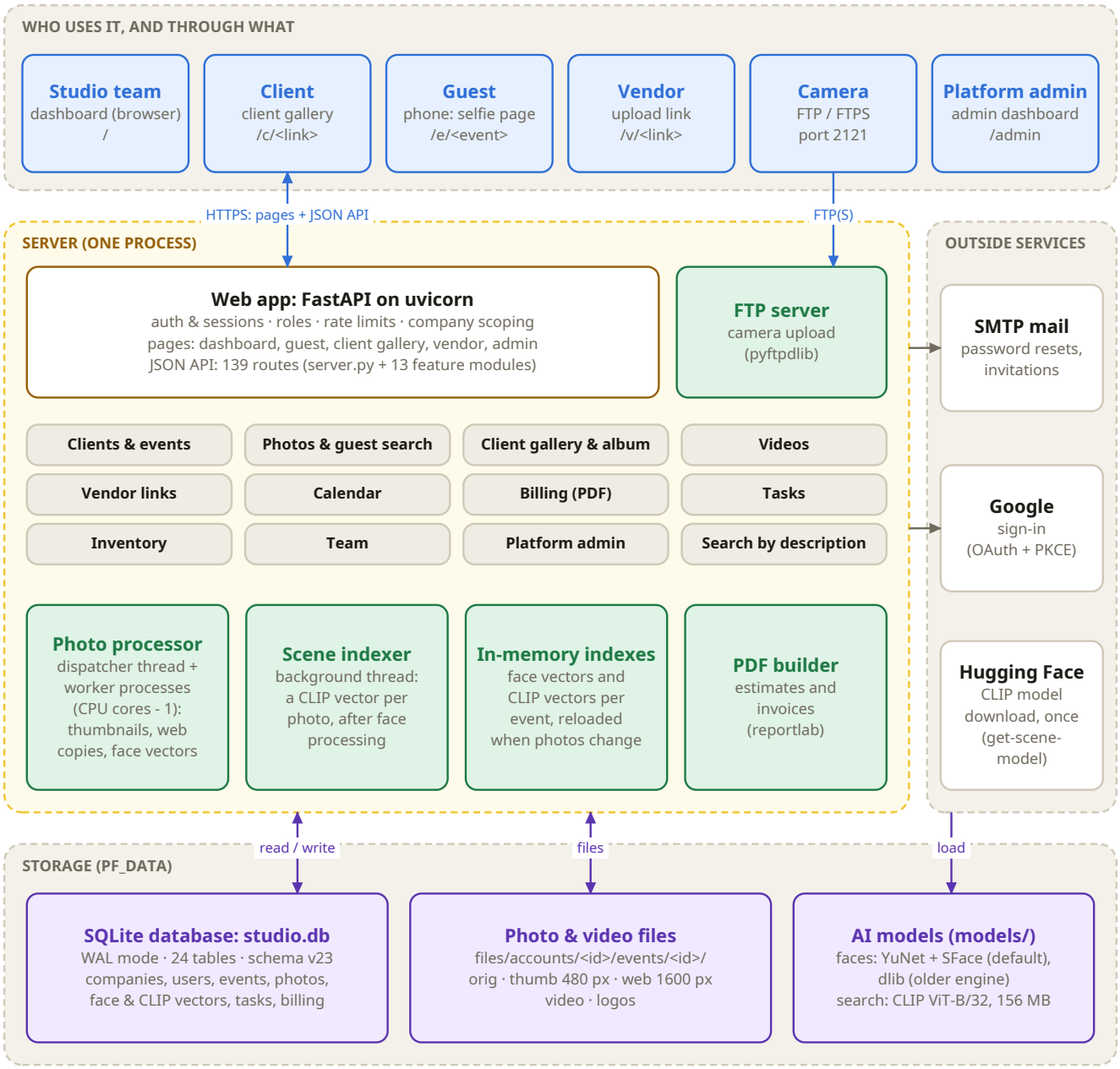


Figure 1. System architecture: users, the server's parts, outside services and storage.

3.1 Modules

Module (studio/)	Responsibility	Routes
server.py	Creates the app; auth, sessions, accounts, clients, events, photos, guest page and search; mounts the other modules	47
processing.py	Photo processor: dispatcher thread and worker pool; in-memory face vector index	–
ingest.py	The single path every upload takes: checks, limits, duplicates, storing, queueing	–
client_gallery.py / album.py / videos.py	Client gallery with PIN, zip streaming, find a person; album selection; event videos	11 / 7 / 7
scenes.py	Search by description: CLIP engine, background indexer, search, chips, model download	6
vendors.py / camera.py	Vendor upload links; camera upload over FTP or FTPS	9 / 4
bookings.py / estimates.py	Calendar bookings; estimates and invoices with PDFs	4 / 8

Module (studio/)	Responsibility	Routes
tasks.py / inventory.py	Project board with vendor work; equipment inventory	8 / 9
team.py / roles.py / security.py	Invitations and members; roles and permissions; passwords, sessions, rate limits	8 / – / –
admin.py	Platform admin dashboard and log	11
google.py / mailer.py / branding.py / storage.py / db.py / config.py	Google sign-in; email; logo handling and page branding; file storage; database and migrations; settings and plans	–
faces.py (project root)	Face engines: YuNet + SFace (default) and dlib	–

3.2 Technology

Layer	Technology	Why
Web	FastAPI + uvicorn; pydantic for request validation	Fast, typed request handling; dependency injection for auth and permissions
Pages	HTML + plain JavaScript (no framework, no build step)	Loads fast on phones; nothing to compile
Database	SQLite in WAL mode, one connection per thread	No database server to run; web server and workers can write at the same time
Images	Pillow (+ pillow-heif for iPhone HEIC)	Thumbnails, web copies, logo clean-up
Faces	OpenCV YuNet (detection) + SFace (recognition)	Accurate, free licences, run on CPU
Search by description	CLIP ViT-B/32 (8-bit ONNX) on onnxruntime, tokenizers	Free and local: no photo leaves the server, no per-use cost
PDF	reportlab	Estimate and invoice PDFs (and this document)
Camera upload	pyftpdlib (+ pyopenssl for FTPS)	What professional cameras speak

3.3 Deployment

On a server it runs in Docker behind an HTTPS reverse proxy. The data folder is a volume, so the container can be rebuilt without losing anything.

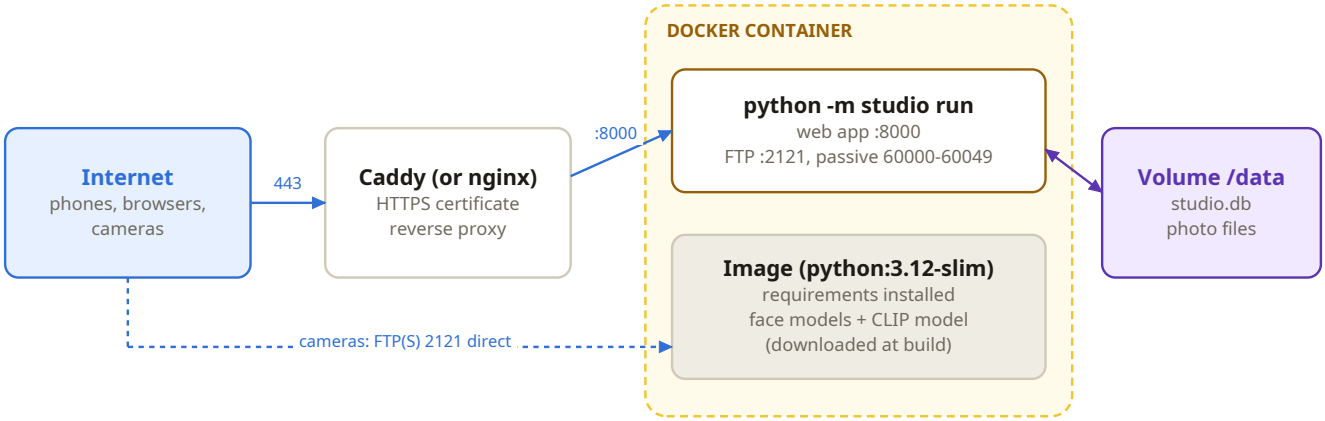


Figure 2. Deployment: HTTPS proxy, the app container and the data volume.

4. How it works

4.1 From upload to searchable photo

Whatever the source, a photo goes through the same steps. The web request only stores the original; everything slow happens in the background, and the database status tells the dashboard where each photo is.

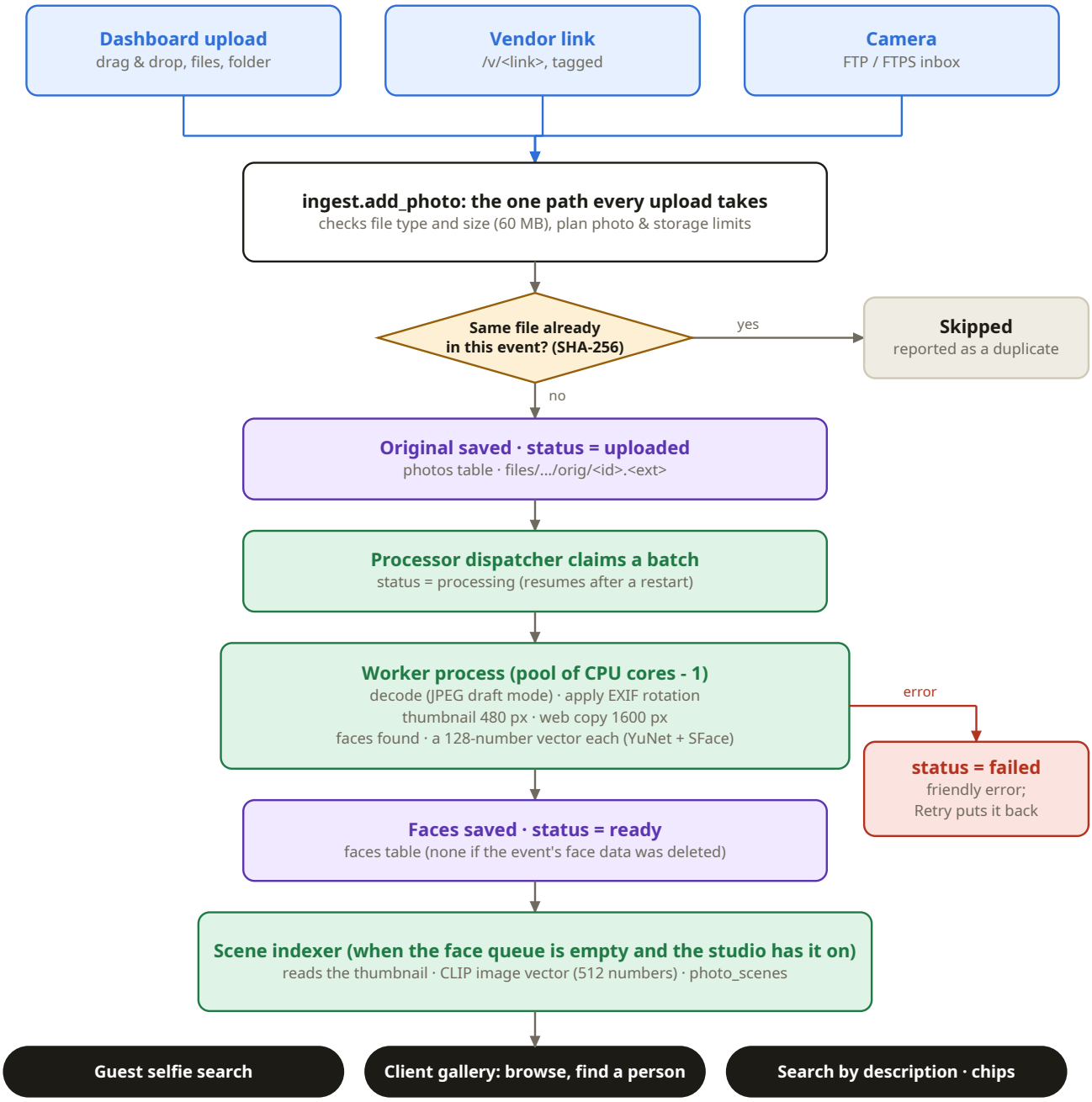


Figure 3. Photo pipeline: upload, checks, processing, face vectors and the CLIP vector.

4.2 A guest finds their photos

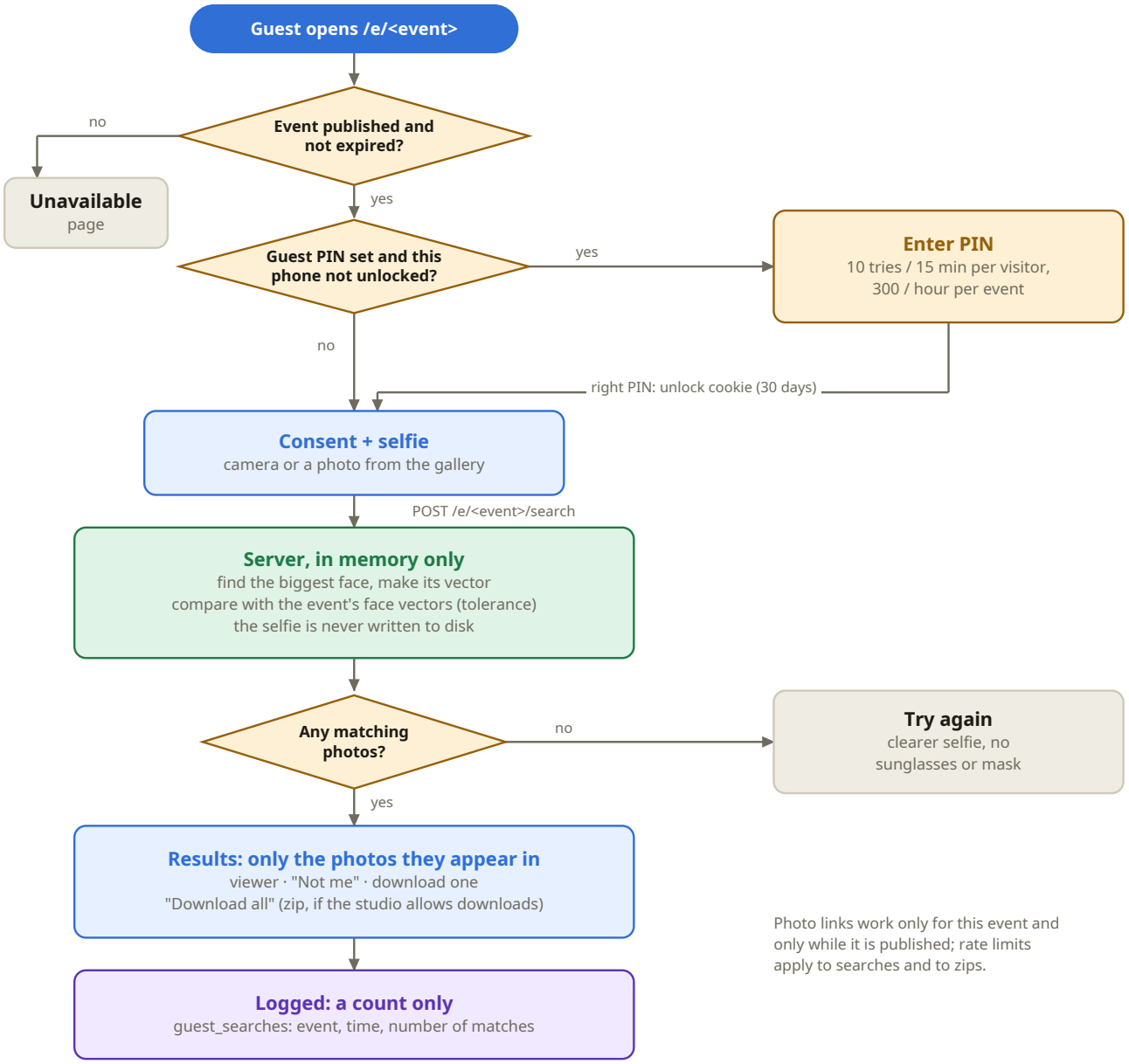
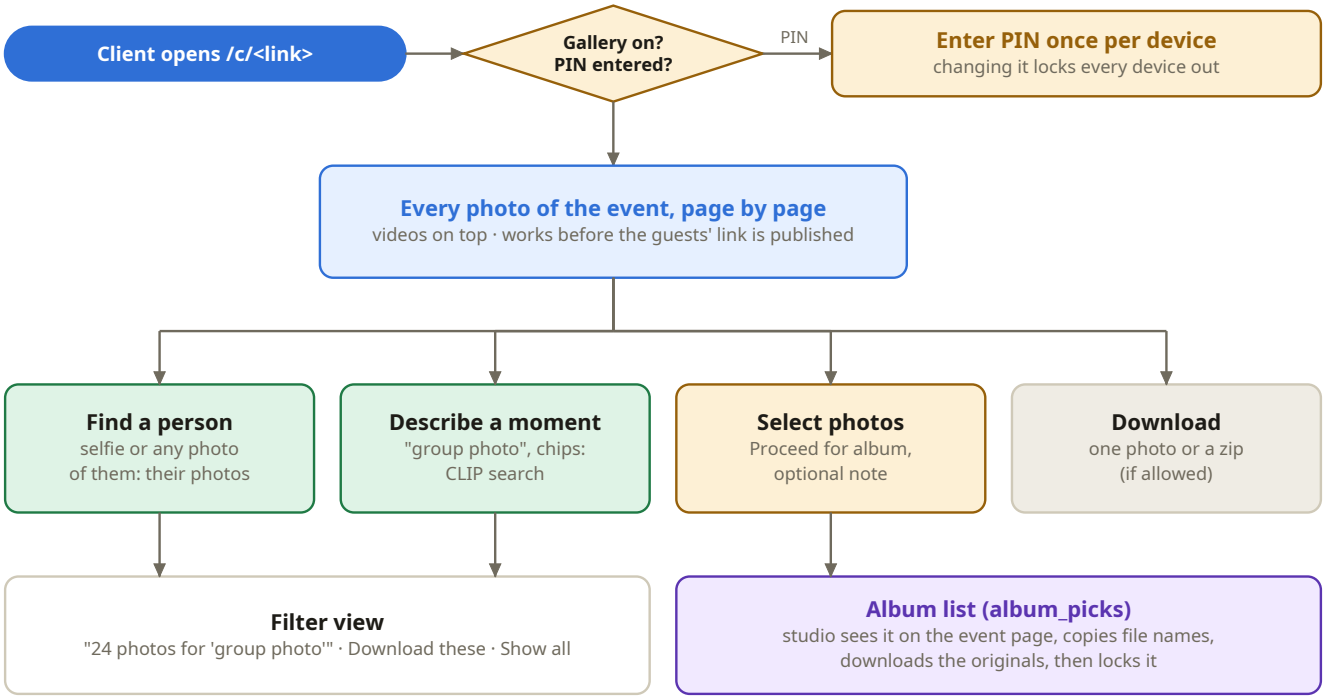


Figure 4. Guest flow on the guest link, with the optional PIN.

4.3 The client gallery



Studio controls (event page): gallery on/off · PIN · client downloads on/off · New link · view count.
Every request carries the link token; the PIN is checked on the server for photos, searches, the album and videos.

Figure 5. Client gallery: browse, find a person, describe a moment, choose album photos.

4.4 Search by description

The CLIP model turns a photo and a sentence into vectors in the same space; the closer they are, the better the photo fits the words. Scores are relative, so a fixed threshold isn't enough. The rule below was measured on a real 101-photo reception: it kept the right photos for "a selfie", "giving a gift" and "couple on stage", and returned nothing for "cake", "dancing", "a dog" and "a car", which weren't there.

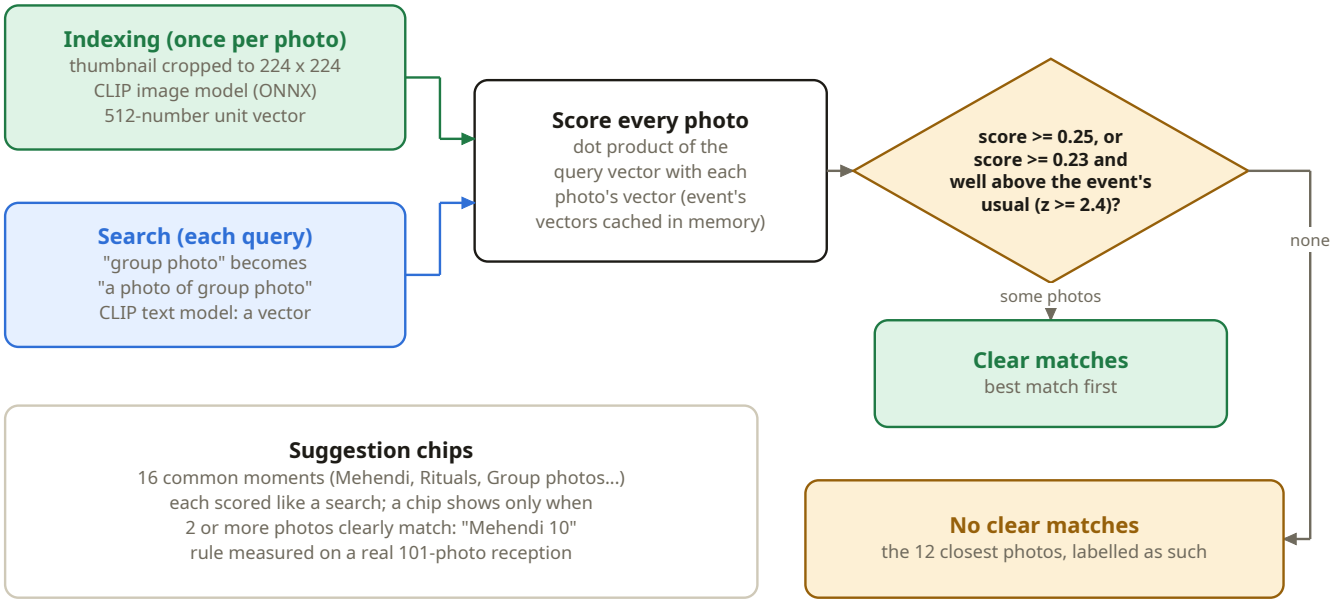
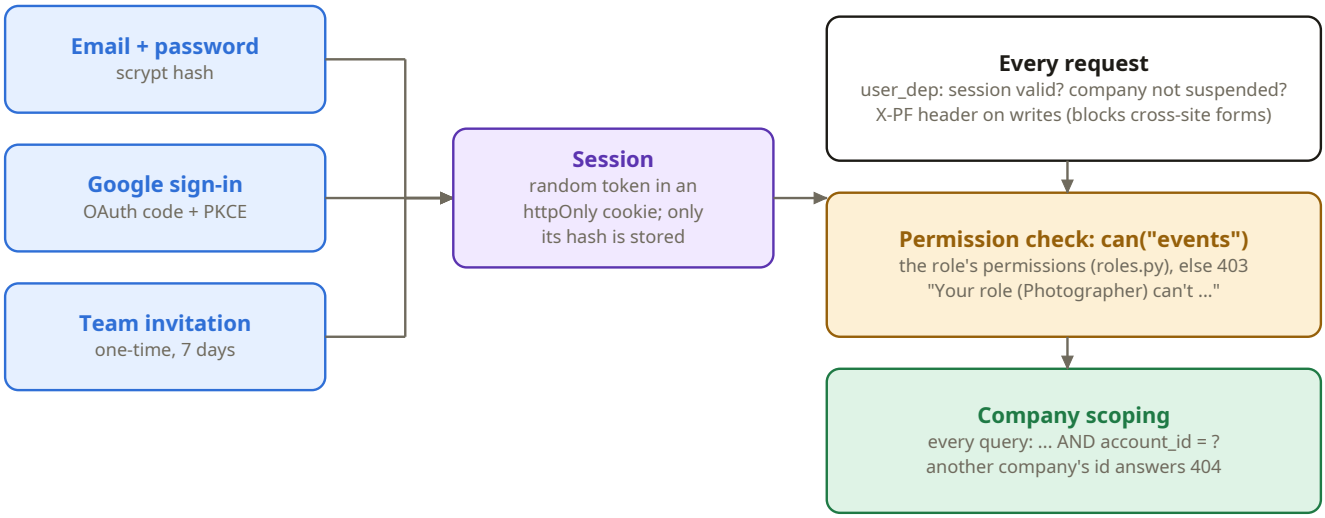


Figure 6. Search by description: indexing, scoring, and the clear-match rule.

4.5 Signing in, roles and company scoping



Public pages (guest /e/, client /c/, vendor /v/) need no account: the unguessable link (plus an optional PIN) is the key.

Figure 7. Every dashboard request: session, permission, company scope.

Permission	Owner	Admin	Manager	Photographer	Viewer
Team (invite, roles)	yes	yes			
Company settings, search switch	yes	yes			
Clients	yes	yes	yes		
Events (create, publish, share, PINs)	yes	yes	yes		
Upload photos	yes	yes	yes	yes	
Delete photos	yes	yes	yes		
Inventory (manage)	yes	yes	yes	own check-out	
Tasks (manage)	yes	yes	yes	tasks they're on	
Delete the company	yes				
See everything in the company	yes	yes	yes	yes	yes

Table: what each role may do (studio/roles.py). The server checks this on every request; the dashboard hides what a role can't use.

4.6 The tasks board

Drag a card between columns (or change the stage in the task). Done and Cancelled are folded at the side.

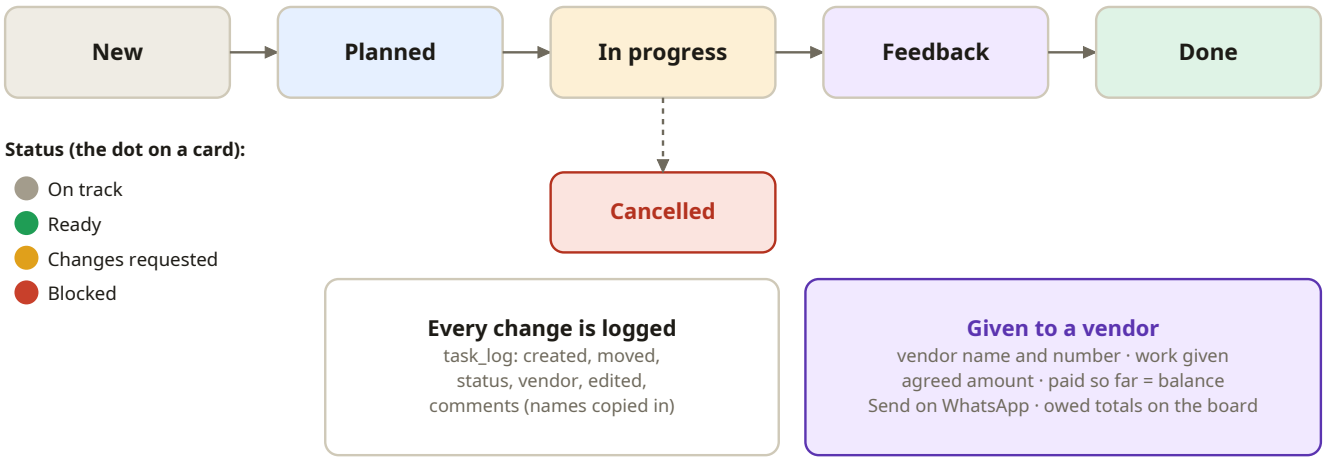
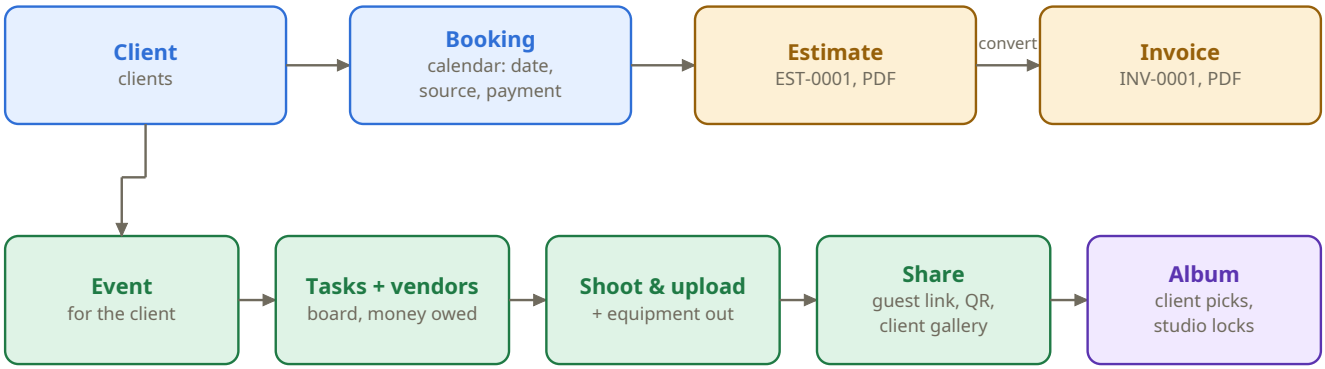


Figure 8. Task stages, status and vendor work.

4.7 The studio's day-to-day workflow



The Overview page pulls these together: late tasks, overdue equipment, upcoming bookings, money to collect and owed.

Figure 9. From client to invoice, and from event to album.

5. Database design

One SQLite file, studio.db, holds everything except the photo and video files. The schema is created at first start and then brought up to date by numbered migrations (studio/db.py); the database records its version (now 23), so each migration runs once. Foreign keys are enforced. Money is stored as whole paise, so totals never pick up rounding errors.

5.1 Entity-relationship diagrams

PK marks a primary key and FK a foreign key; N, 1 and 0..1 say how many rows can be on each end of a line. Columns that are listed together ("name, email, phone") are separate columns, grouped to keep the diagrams small; the data dictionary in 5.3 lists every column.

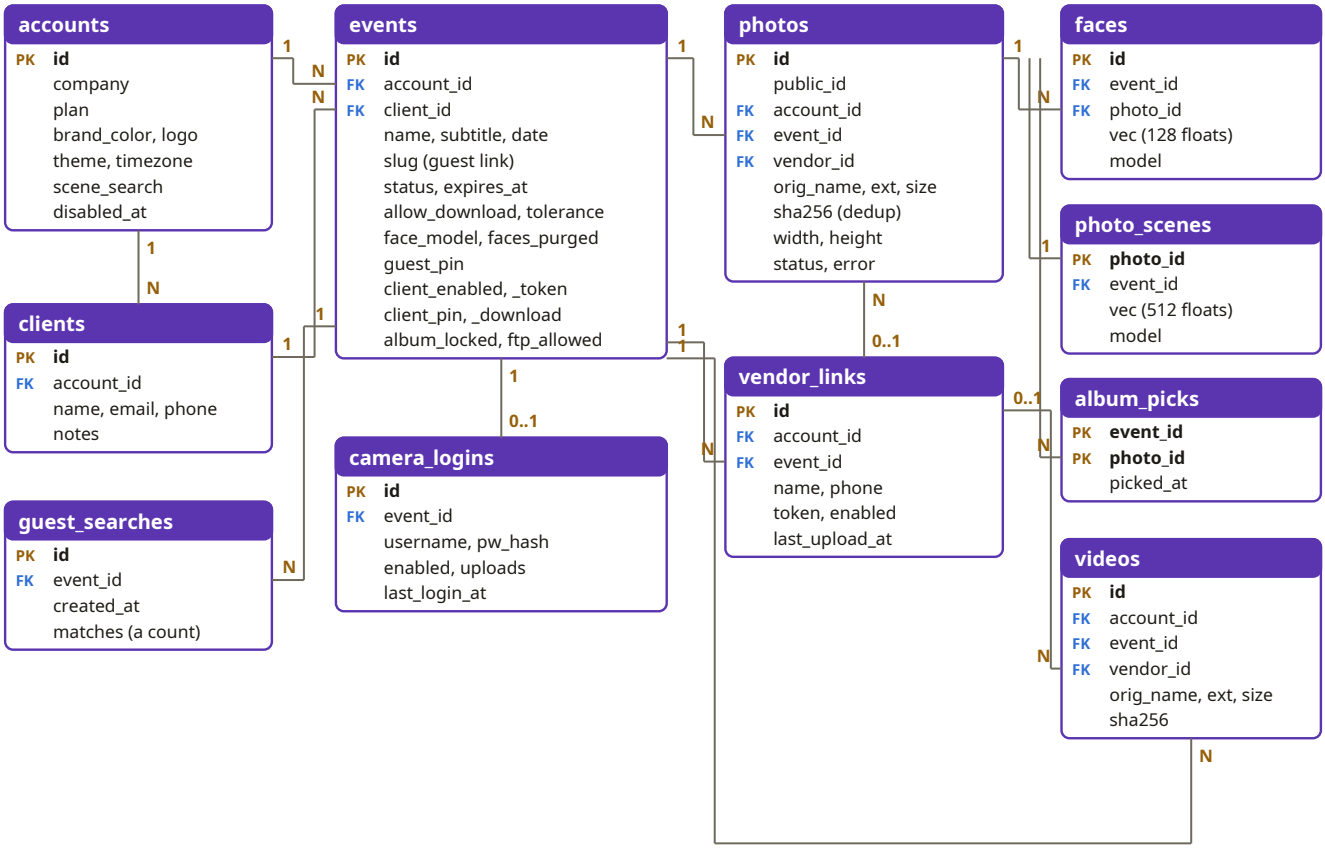


Figure 10. Core: companies, clients, events, photos and everything attached to them.

Every photo-side table hangs off an event, and every event off a company. Deleting an event deletes its photos, face and CLIP vectors, album picks, videos, vendor links and camera login (ON DELETE CASCADE). A client can't be deleted while it has events (RESTRICT). Deleting a vendor link keeps the photos it brought, untagged (SET NULL). photos, videos and vendor_links also carry account_id (lines to accounts left out for readability).

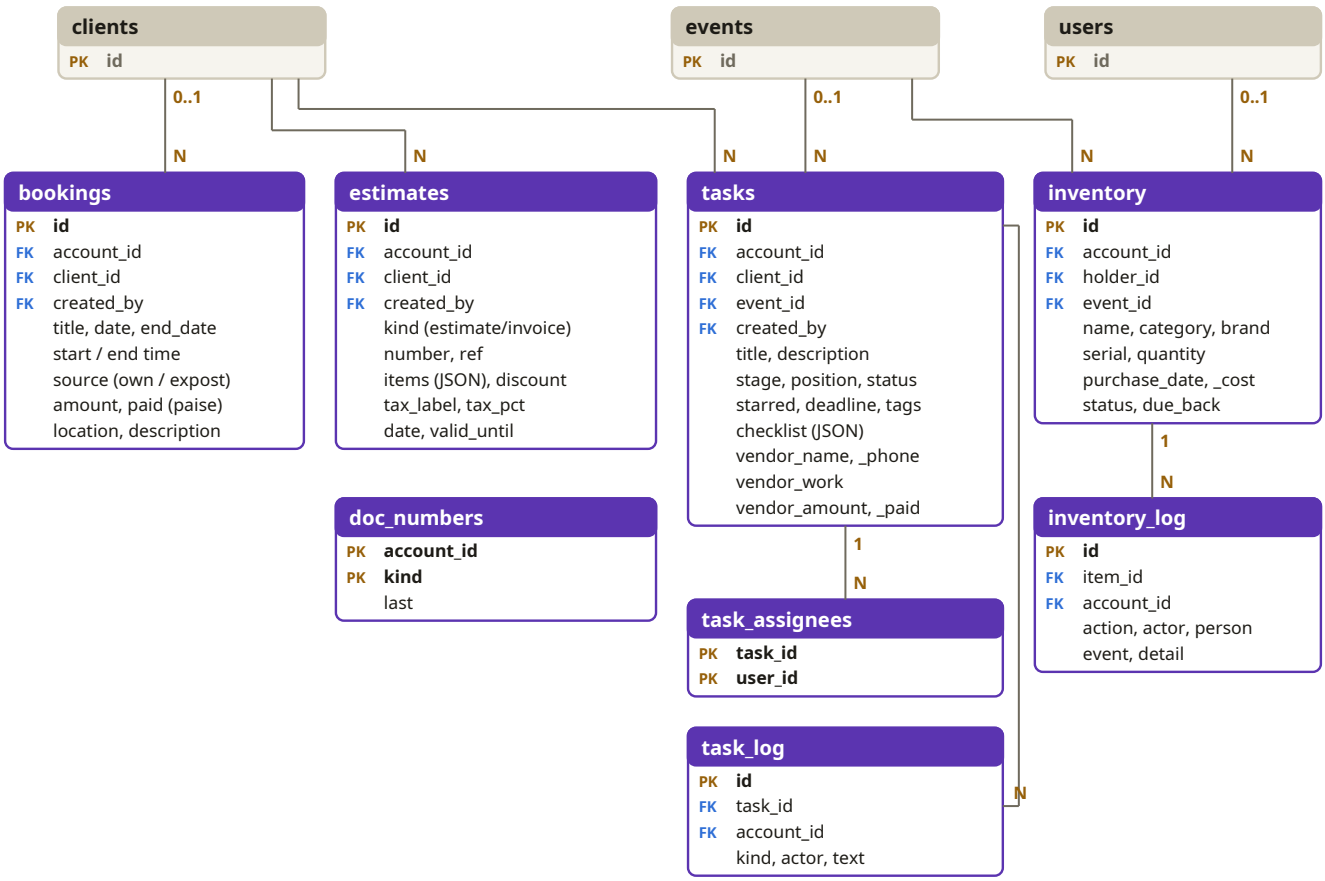


Figure 11. Business operations: bookings, billing, tasks and inventory. Grey boxes are tables from Figure 10 and 12.

Every table here also has account_id (company scoping). created_by on bookings, estimates and tasks, and task_assignees.user_id, point to users (lines left out for readability). doc_numbers holds the last estimate and invoice number per company, so deleting the newest document can't hand its number out again. The logs copy names in, so the history still reads right after people or events are deleted.

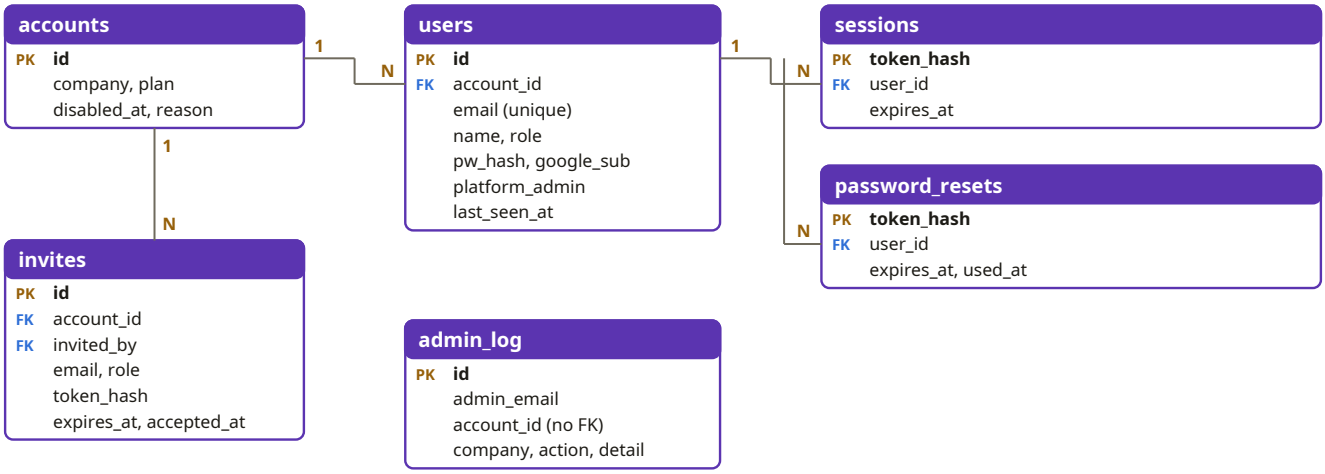


Figure 12. Identity and administration. admin_log keeps no foreign key on purpose: the record survives the company's deletion.

5.2 Design rules

- **Multi-company (tenant) isolation:** every company-owned table has account_id, and every dashboard query filters on it. A test fails if that scoping is removed.
- **Deletion follows ownership:** deleting a company deletes everything it owns (CASCADE). Optional links (a task's event, a booking's client) become empty (SET NULL) rather than blocking.
- **Secrets are never stored in plain form:** passwords and PINs are scrypt hashes; session, reset and invitation tokens are SHA-256 hashes.
- **Vectors are binary blobs:** faces.vec (128 float32) and photo_scenes.vec (512 float32), with the model name, so changing a model can re-index without mixing results.
- **Histories outlive what they mention:** task_log, inventory_log and admin_log copy names in.
- **Lists stored as JSON** where they are always read together: estimate line items, task tags and checklists.

5.3 Data dictionary

Generated from the live schema (version 23). Notes come from the comments in studio/db.py.

Identity and administration

accounts A photography company (the tenant). Plan, branding, time zone, suspension, search-by-description switch.

Column	Type	Key	Notes
id	integer	PK	
company	text		
brand_color	text		
plan	text		
created_at	integer		
timezone	text		
disabled_at	integer, null		set = suspended by the product owner
disabled_reason	text		
logo	text		random file name; '' = show the name
logo_size	text		small | medium | large
logo_badge	integer		too dark for the dark pages: light badge
theme	text		guest/client pages: dark | light | auto
logo_badge_light	integer		too light for the light pages
scene_search	integer		the studio's on/off switch

users People who sign in. Each belongs to one company and has a role.

Column	Type	Key	Notes
id	integer	PK	
account_id	integer	FK	references accounts.id (cascade)
email	text		
name	text		
pw_hash	text		
role	text		
created_at	integer		
platform_admin	integer		the product owner (python -m studio grant-admin)
last_seen_at	integer, null		
google_sub	text, null		Google account id ("sub"), set once linked; pw_hash '' = no password

sessions Signed-in browsers. Only a hash of the cookie token is stored.

Column	Type	Key	Notes
token_hash	text	PK	
user_id	integer	FK	references users.id (cascade)
expires_at	integer		
created_at	integer		

password_resets One-time password reset links (hashed), valid 60 minutes.

Column	Type	Key	Notes
token_hash	text	PK	
user_id	integer	FK	references users.id (cascade)
expires_at	integer		
used_at	integer, null		
created_at	integer		

invites Team invitations by email (hashed token), valid 7 days.

Column	Type	Key	Notes
id	integer	PK	
account_id	integer	FK	references accounts.id (cascade)
email	text		
name	text		
role	text		
token_hash	text		
invited_by	integer, null	FK	references users.id (set null)
created_at	integer		
expires_at	integer		
accepted_at	integer, null		

admin_log Everything the platform admin did. Kept even after a company deletes itself.

Column	Type	Key	Notes
id	integer	PK	
admin_user_id	integer, null		
admin_email	text		
account_id	integer, null		
company	text		
action	text		
detail	text		
created_at	integer		

Clients, events and photos

clients The couples, companies and schools a studio shoots for.

Column	Type	Key	Notes
id	integer	PK	
account_id	integer	FK	references accounts.id (cascade)
name	text		
email	text		
phone	text		
notes	text		
created_at	integer		

events One shoot for a client: its guest link, client gallery, PINs, album and camera settings.

Column	Type	Key	Notes
id	integer	PK	
account_id	integer	FK	references accounts.id (cascade)
client_id	integer	FK	references clients.id (restrict)
name	text		
subtitle	text		
event_date	text, null		
slug	text		
status	text		
allow_download	integer		
expires_at	text, null		
tolerance	real		
faces_purged	integer		
created_at	integer		
face_model	text		which engine the event searches with
client_enabled	integer		the all-photos link for the client
client_token	text, null		the secret in /c/<token>
client_pin	text		scrypt hash of the optional PIN
client_download	integer		
client_views	integer		
ftp_allowed	integer		camera upload, switched on by the platform admin
album_note	text		
album_sent_at	integer, null		
album_locked	integer		
guest_pin	text		scrypt hash of the guests' optional PIN

photos Every uploaded photo and its processing status.

Column	Type	Key	Notes
id	integer	PK	
public_id	text		
account_id	integer	FK	references accounts.id (cascade)
event_id	integer	FK	references events.id (cascade)
orig_name	text		
ext	text		
size	integer		
sha256	text		
width	integer, null		
height	integer, null		
status	text		
error	text, null		
created_at	integer		
vendor_id	integer, null	FK	references vendor_links.id (set null)

faces One face vector per face found in a photo (biometric; deletable per event).

Column	Type	Key	Notes
id	integer	PK	
event_id	integer	FK	references events.id (cascade)
photo_id	integer	FK	references photos.id (cascade)
vec	blob		
model	text		which engine made the vector

photo_scenes One CLIP vector per photo, for search by description.

Column	Type	Key	Notes
photo_id	integer	PK, FK	references photos.id (cascade)
event_id	integer	FK	references events.id (cascade)
vec	blob		512 float32; empty = the thumbnail couldn't be read
model	text		

guest_searches A count of guest selfie searches: when, and how many photos matched. No image.

Column	Type	Key	Notes
id	integer	PK	
event_id	integer	FK	references events.id (cascade)
created_at	integer		
matches	integer		

album_picks Photos the client chose for the album.

Column	Type	Key	Notes
event_id	integer	PK, FK	references events.id (cascade)
photo_id	integer	PK, FK	references photos.id (cascade)
picked_at	integer		

vendor_links Upload links for second shooters, drone and video teams.

Column	Type	Key	Notes
id	integer	PK	
account_id	integer	FK	references accounts.id (cascade)
event_id	integer	FK	references events.id (cascade)
name	text		
phone	text		
token	text		
enabled	integer		
created_by	integer, null	FK	references users.id (set null)
created_at	integer		
last_upload_at	integer, null		

videos Event videos, shown in the client gallery only.

Column	Type	Key	Notes
id	integer	PK	
public_id	text		
account_id	integer	FK	references accounts.id (cascade)
event_id	integer	FK	references events.id (cascade)
orig_name	text		
ext	text		
size	integer		
sha256	text		
created_at	integer		
vendor_id	integer, null	FK	references vendor_links.id (set null)

camera_logins One FTP login per event for camera upload.

Column	Type	Key	Notes
id	integer	PK	
event_id	integer	FK	references events.id (cascade)
username	text		
pw_hash	text		
enabled	integer		
created_at	integer		
last_login_at	integer, null		
last_upload_at	integer, null		
uploads	integer		
last_note	text		

Business operations

bookings Calendar bookings with what was agreed and paid.

Column	Type	Key	Notes
id	integer	PK	
account_id	integer	FK	references accounts.id (cascade)
title	text		
date	text		first day, YYYY-MM-DD
end_date	text, null		last day of a multi-day booking
start_time	text, null		HH:MM
end_time	text, null		
source	text		own (own client) | expost
client_id	integer, null	FK	references clients.id (set null)
client_name	text		
phone	text		
location	text		
description	text		
amount	integer		finalised, paise
paid	integer		received so far, paise
payment_note	text		
created_by	integer, null	FK	references users.id (set null)
created_at	integer		
updated_at	integer		

estimates Estimates (quotations) and invoices, with line items as JSON.

Column	Type	Key	Notes
id	integer	PK	
account_id	integer	FK	references accounts.id (cascade)
kind	text		estimate (EST-0001) | invoice (INV-0001)
number	integer		per company: EST-0001, EST-0002, …
ref	text		an invoice made from an estimate: that estimate's number
client_id	integer, null	FK	references clients.id (set null)
company_name	text		printed on it; defaults to the company's name
company_details	text		address, phone, email, GSTIN
show_logo	integer		
client_name	text		
client_details	text		
event_title	text		
estimate_date	text		the document's date (estimate date or invoice date)
valid_until	text, null		estimate: valid until; invoice: due date
currency	text		
items	text		JSON [{name, description, amount}], amounts in paise
discount	integer		paise
tax_label	text		
tax_pct	real		
notes	text		
created_by	integer, null	FK	references users.id (set null)
created_at	integer		
updated_at	integer		

doc_numbers The last estimate / invoice number given out, per company.

Column	Type	Key	Notes
account_id	integer	PK, FK	references accounts.id (cascade). the newest one can't free it
kind	text	PK	
last	integer		

tasks Cards on the project board, including work given to vendors.

Column	Type	Key	Notes
id	integer	PK	
account_id	integer	FK	references accounts.id (cascade)
title	text		
description	text		
stage	text		new | planned | progress | feedback | done | cancelled
position	real		order inside the stage's column
status	text		on_track | ready | changes | blocked
starred	integer		
deadline	text, null		YYYY-MM-DD
tags	text		JSON ["Post-prod", "Onsite"]
client_id	integer, null	FK	references clients.id (set null)
event_id	integer, null	FK	references events.id (set null)
checklist	text		JSON [{text, done}]
vendor_name	text		'' = done in-house
vendor_phone	text		
vendor_work	text		the work given to the vendor
vendor_amount	integer		agreed, paise
vendor_paid	integer		paid so far, paise
created_by	integer, null	FK	references users.id (set null)
created_at	integer		
updated_at	integer		

task_assignees Which team members are on which task.

Column	Type	Key	Notes
task_id	integer	PK, FK	references tasks.id (cascade)
user_id	integer	PK, FK	references users.id (cascade)

task_log Comments and changes on each task.

Column	Type	Key	Notes
id	integer	PK	
task_id	integer	FK	references tasks.id (cascade)
account_id	integer	FK	references accounts.id (cascade)
kind	text		comment | created | moved | status | vendor | vendor_off | edited
actor	text		
text	text		
created_at	integer		

inventory The studio's equipment and who has it.

Column	Type	Key	Notes
id	integer	PK	
account_id	integer	FK	references accounts.id (cascade)
name	text		
category	text		
brand	text		
serial	text		
quantity	integer		
purchase_date	text, null		
purchase_cost	integer		paise
notes	text		
status	text		available | out | maintenance | retired
holder_id	integer, null	FK	references users.id (set null)
event_id	integer, null	FK	references events.id (set null)
checked_out_at	integer, null		
due_back	text, null		
created_at	integer		
updated_at	integer		

inventory_log History of each equipment item.

Column	Type	Key	Notes
id	integer	PK	
item_id	integer	FK	references inventory.id (cascade)
account_id	integer	FK	references accounts.id (cascade)
action	text		added | edited | checked_out | returned | maintenance | available | retired
actor	text		
person	text		
event	text		
detail	text		
created_at	integer		

6. Security and privacy

- Passwords: scrypt. Sessions: random tokens stored hashed, httpOnly cookie, 14 days, other devices signed out on a password change.
- Password reset: one-time links valid 60 minutes, stored hashed; the same answer whether or not an email exists; at most 3 emails per address per hour.
- Google sign-in: authorization code with PKCE, state bound to the browser, nonce, and the ID token's issuer, audience, expiry and verified email checked.
- Rate limits on sign-in and sign-up, guest searches, client searches, PIN tries (10 per 15 minutes per visitor) and Download all.

- Every dashboard request is scoped to the signed-in company; another company's ids answer 404. A request-header check blocks cross-site form posts.
- Links are unguessable: guest links have random slugs, client gallery and vendor links 128-bit tokens. Unpublished or expired events are fully offline.
- Selfies are processed in memory and never stored; only a count of searches is kept. Face vectors can be deleted per event; a company can delete its whole account.
- All AI runs on the studio's server: face matching and search by description never send a photo to an outside service.
- Uploaded logos are redrawn as a fresh PNG (no metadata or hidden payload). All user text is escaped on every page.

7. API overview

All routes return JSON except pages, images, zips and PDFs. Dashboard routes need a session; public routes use their link token.

Group	Path prefix	Who	Module
Auth and account	/api/auth/*, /api/account*	anyone / signed in	server.py, google.py
Clients and events	/api/clients, /api/events/{id}	signed in, by role	server.py
Photos	/api/events/{id}/photos*	signed in, by role	server.py
Guest page	/e/{slug}, /info, /unlock, /search, /thumb, /web, /download, /zip	public (+ PIN)	server.py
Client gallery	/c/{token}/info, /unlock, /photos, /search, /zip, /album, /videos, /scenes, /describe	public (+ PIN)	client_gallery.py, album.py, videos.py, scenes.py
Vendor upload	/v/{token}, /info, /photos, /videos	public link	vendors.py
Studio features	/api/calendar, /api/estimates, /api/tasks, /api/inventory, /api/team, /api/scenes	signed in, by role	bookings, estimates, tasks, inventory, team, scenes
Event extras	/api/events/{id}/vendors, /camera, /album, /videos, /scenes, /describe, /guest-pin, /client-pin	signed in, by role	several
Platform admin	/api/admin/*	platform admin (others get 404)	admin.py
Health	/healthz	anyone	server.py

8. Testing

Automated tests use pytest with FastAPI's test client against a fresh data folder per test, so they never touch real data. 174 tests pass; 2 are skipped unless a folder of sample faces is given. Page scripts are checked with Node.js so a syntax error can't blank a page.

Test file	What it proves
test_studio.py, test_regressions.py	Sign-up, clients, events, uploads, publishing, guest search; company isolation
test_face_engine.py, test_face_flow.py	Face engines and the confirmation rule; end-to-end with real faces (needs sample photos)
test_client_gallery.py, test_album.py, test_videos.py	Client gallery, PIN, zips, find a person, album selection, videos
test_guest_pin.py	Guest PIN: nothing leaks before it, changing it locks people out, guessing is limited
test_scenes.py	Search by description: indexing, search, chips, switch, roles, deletion, the clear-match rule, and the real CLIP model

Test file	What it proves
test_tasks.py, test_inventory.py, test_calendar.py, test_estimates.py	Board and vendor money, equipment, bookings, estimates and invoices
test_team.py, test_google.py, test_reset.py, test_admin.py	Roles and invitations, Google sign-in, password reset, platform admin
test_vendors.py, test_camera.py, test_branding.py, test_theme.py, test_pages.py	Vendor links, FTP upload, logos, themes, page scripts

9. Running it

Command	What it does
pip install -r requirements.txt	Install the Python packages
python -m studio get-scene-model	Download the search-by-description model once (156 MB, checksums verified)
python -m studio run [--port 8000] [--public-url ...] [--ftp-port 2121]	Start the app; prints the dashboard address
python -m studio create-account --company ... --email ...	Create a company from the terminal
python -m studio set-plan <email> pro	Move a company to another plan
python -m studio reset-password <email>	Set a new password for someone locked out
python -m studio grant-admin <email>	Give someone the platform admin dashboard
docker compose up -d --build	Run on a server with Docker

Setting (environment)	Meaning
PF_DATA	Data folder (database and files)
PF_PUBLIC_URL	The address used in share links, e.g. https://photos.example.com
PF_ALLOW_SIGNUP=0	Close public sign-up
PF_SMTP_HOST, _PORT, _USER, _PASSWORD, _SECURITY, PF_MAIL_FROM	Outgoing email
PF_GOOGLE_CLIENT_ID, PF_GOOGLE_CLIENT_SECRET	Google sign-in
PF_FTP_PORT, PF_FTP_PASSIVE, PF_FTP_CERT, PF_FTP_KEY	Camera upload
PF_TRUST_PROXY	Whether to read the visitor address from a reverse proxy (auto, 1, 0)

10. Known limitations and next steps

- One process with SQLite: fine for a few dozen active studios; beyond that, PostgreSQL, object storage and separate workers.
- Search by description understands English best and scenes better than relationships; its match rule was tuned on one event.
- No online payments or plan billing yet; plans are changed by the platform admin.
- Terms of service and a privacy policy are needed before selling: face vectors are biometric data (India's DPDP Act, GDPR).
- The Docker image is untested where it was written; build and test it before relying on it.
- Next features already designed on top of the stored CLIP vectors: More like this, hiding near-duplicate shots, chapters in the client gallery, and a free album shortlist.